

A BUILD-BY-DOING TUTORIAL

FastAPI Zero to Production

From “what is an API?” to a deployed, tested, authenticated backend — one phase at a time.

2

PHASE TWO

Path Params, Query Params & Request Body

Make endpoints dynamic: send data into your API through the path, query parameters, and the request body — then build the first working version of the Book Library API.

Contents

2.1 Path Parameters	5
Automatic type validation	5
Restricting to a fixed set with Enum	6
Multiple path parameters	6
Order matters — a classic bug	6
2.2 Query Parameters	7
Required vs optional query parameters	8
Boolean query parameters	8
List query parameters	9
2.3 Request Body	9
Why not just use query params for everything?	10
Receiving a body: raw dict vs Pydantic model	10
Combining path + query + body in one endpoint	11
2.4 Headers and Cookies	11
Reading headers with Header()	12
Reading cookies with Cookie()	12
2.5 Response Customisation	12
Different response types	13
Returning files and streaming	13
Hands-On Exercise 2: Build the Book Library API (in-memory)	14
Checkpoint: At This Point You Can Now...	18
Phase 2 Summary	19

Your Journey

✓	Foundations: What Is an API?	DONE
✓	Setup & Your First API	DONE
2	Path Params, Query Params & Body	YOU ARE HERE
3	Pydantic: Data Validation	
4	Database Integration (SQLAlchemy)	
5	Authentication & Authorization	
6	Advanced FastAPI Features	
7	Testing Your FastAPI App	
8	Deployment with Docker	
★	Capstone — TaskFlow API	

In Phase 1 your endpoints were like vending machines with one button: press it, get the same snack every time. Real APIs respond to *input* — "give me user **42**", "show me books in the **scifi** genre", "create a book with **these details**". There are exactly three channels for that input, and the whole of this phase is learning when to use each:

1. **Path parameters** — part of the URL itself, used to identify *which* resource. (`/books/42`)
2. **Query parameters** — the `?key=value` tail of a URL, used to filter, sort, paginate. (`/books?genre=scifi`)
3. **Request body** — a JSON payload sent with the request, used for the actual data of a new or updated resource.

Let's take them one at a time.

2.1 Path Parameters

A **path parameter** is a variable slot inside the URL path. You use it to point at one specific resource. Remember the RESTful URLs from Phase 0: `/books/42` means "the book whose id is 42." That `42` is a path parameter.

You declare it with curly braces in the path, and as a function argument with the same name:

PYTHON

```
from fastapi import FastAPI

app = FastAPI()

# {book_id} is a path parameter. FastAPI passes it to the function argument
# of the same name. The type hint `int` makes FastAPI convert and validate it.
@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id, "you_requested": f"book number {book_id}"}
```

Visit `http://127.0.0.1:8000/books/42` and you get `{"book_id": 42, "you_requested": "book number 42"}`.

Automatic type validation

Because you typed `book_id: int`, FastAPI tries to convert the URL segment to an integer. Watch what happens with good and bad input:

TEXT

GET /books/42	→ 200 OK	{"book_id": 42, ...}	(converted to int 42)
GET /books/hello	→ 422	{"detail": [...]}	(can't convert "hello" to int)

You wrote zero validation code, yet `/books/hello` is rejected with a helpful `422` explaining that an integer was expected. That's the type hint doing the work.

TIP

Tip: FastAPI supports several path-parameter types out of the box: `int`, `float`, `str`, `bool`, and `UUID` (for universally-unique identifiers like `a1b2c3d4-...`). Pick the type that matches your real data, and you get free validation.

Restricting to a fixed set with Enum

Sometimes a path parameter should only be one of a few known values — say, a model name or a category. Use a Python `Enum` and FastAPI will reject anything else *and* list the valid options in the docs:

PYTHON

```
from enum import Enum
from fastapi import FastAPI

app = FastAPI()

# Subclassing str + Enum makes the values both real strings and a closed set.
class Genre(str, Enum):
    scifi = "scifi"
    fantasy = "fantasy"
    history = "history"

@app.get("/genres/{genre}")
def read_genre(genre: Genre):
    # genre is one of the Enum members; .value gives the underlying string.
    return {"genre": genre.value, "message": f"You picked {genre.value}"}
```

Now `/genres/scifi` works, but `/genres/romance` returns a `422` listing the allowed values. The Swagger docs even render a dropdown.

Multiple path parameters

You can have several in one path. They're matched left to right:

PYTHON

```
# Two path parameters: a user id and one of their book ids.
@app.get("/users/{user_id}/books/{book_id}")
def get_user_book(user_id: int, book_id: int):
    return {"user_id": user_id, "book_id": book_id}
```

Order matters — a classic bug

FastAPI matches routes **in the order you define them**, top to bottom. If a fixed path looks like a parameterised one, the parameterised route can "swallow" it. Watch:

PYTHON

```
# WRONG ORDER
@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id}

@app.get("/books/featured") # this can never be reached!
def featured():
    return {"featured": True}
```

A request for `/books/featured` hits the **first** route, FastAPI tries to convert `"featured"` to an `int`, and you get a `422` — the `featured` endpoint never runs. The fix is simply to **declare fixed/specific routes before parameterised ones**:

PYTHON

```
# CORRECT ORDER – specific route first
@app.get("/books/featured")
def featured():
    return {"featured": True}

@app.get("/books/{book_id}")
def get_book(book_id: int):
    return {"book_id": book_id}
```

COMMON MISTAKES

Common Mistakes: This ordering bug is one of the most common early FastAPI confusions. If a route "isn't being called" or you get an unexpected `422` on a word you expected to work, check whether a `{parameter}` route above it is catching the request first.

CONCEPT CHECK

Concept Check: Should a *date filter* like "books published after 2000" be a path parameter? Answer: No. Path parameters identify a specific resource ("which book"). A filter that narrows a list is a query parameter — that's section 2.2.

2.2 Query Parameters

A **query parameter** is the `?key=value` part at the end of a URL. Multiple are joined with `&`: `/books?genre=scifi&limit=10`. You use them to **filter, sort, paginate, or search** — anything that modifies *how* you retrieve a collection without identifying one specific item.

Here's the beautiful part: in FastAPI, **any function argument that is *not* in the path is automatically treated as a query parameter**.

PYTHON

```
from fastapi import FastAPI

app = FastAPI()

# `genre` and `limit` aren't in the path, so they're read from the query string.
# Defaults make them optional: /books works, /books?genre=scifi&limit=5 works too.
@app.get("/books")
def list_books(genre: str = "all", limit: int = 10):
    return {"genre": genre, "limit": limit}
```

TEXT

GET /books	→ {"genre": "all", "limit": 10}	(defaults used)
GET /books?genre=scifi	→ {"genre": "scifi", "limit": 10}	
GET /books?genre=scifi&limit=5	→ {"genre": "scifi", "limit": 5}	

Required vs optional query parameters

- Give a parameter a **default value** → it's **optional**.
- Give it **no default** → it's **required** (omitting it returns 422).
- To allow "not provided at all," use `str | None = None` (the modern Python 3.11 way to say "a string or nothing").

PYTHON

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/search")
def search(
    q: str,                                # required: /search with no q → 422
    sort: str = "title",                  # optional with a default
    author: str | None = None,            # truly optional; None if omitted
):
    return {"q": q, "sort": sort, "author": author}
```

Boolean query parameters

FastAPI is friendly about booleans — it accepts `true/false`, `1/0`, `yes/no`, `on/off` (case-insensitive) and converts them to a real Python `bool`:

PYTHON

```
@app.get("/books-bool")
def list_books_bool(available: bool = False):
    return {"available": available}
```

`/books-bool?available=true`, `?available=1`, and `?available=yes` all give `{"available": true}`.

List query parameters

To accept the *same* key multiple times (`?tag=python&tag=api`), type the parameter as a `list`. You need FastAPI's `Query` helper to make this work:

PYTHON

```
from fastapi import FastAPI, Query

app = FastAPI()

# Query(default=[]) tells FastAPI to collect every ?tag=... into a list.
@app.get("/items")
def list_items(tag: list[str] = Query(default=[])):
    return {"tags": tag}
```

`/items?tag=python&tag=api` returns `{"tags": ["python", "api"]}`.

TIP

Tip: Use `Query()` whenever you want to add rules or metadata to a query parameter — a minimum/maximum length, a description for the docs, or a pattern. Example: `q: str = Query(min_length=3, max_length=50, description="Search term")`. These show up in `/docs` automatically and are validated for you.

CONCEPT CHECK

Concept Check: A client calls `/books?limit=abc`. What happens? Answer: `422 Unprocessable Entity`. You typed `limit: int`, so FastAPI tries to convert `"abc"` to an integer, fails, and returns a clear validation error — no crash.

2.3 Request Body

Path and query parameters live *in the URL*, which is fine for small bits of data. But to **create or update** a resource you usually send a richer chunk of structured data — a whole book with a title, author, year, and genre. That goes in the **request body** as JSON, and it's sent with `POST`, `PUT`, or `PATCH` (never `GET` or `DELETE`).

Why not just use query params for everything?

URLs are short, visible (they appear in logs and browser history), and awkward for nested or large data. A request body can be large, structured, and is the conventional, RESTful place for "here is the data for the thing I'm creating."

Receiving a body: raw dict vs Pydantic model

You can accept a raw dictionary, but you get no validation:

PYTHON

```
from fastapi import FastAPI

app = FastAPI()

# Accepting a plain dict works, but anything goes – no validation, no docs.
@app.post("/books-raw")
def create_book_raw(book: dict):
    return {"received": book}
```

The far better way — and the FastAPI way — is a **Pydantic model**. You define the shape once as a class, and FastAPI parses the JSON, validates every field, and hands you a clean Python object. (Pydantic gets a full phase of its own next; here's a first taste.)

PYTHON

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# A Pydantic model describes the expected JSON shape and types.
class Book(BaseModel):
    title: str
    author: str
    year: int
    genre: str

# Type the body argument as the model. FastAPI does the rest:
# parse JSON → validate types → give you a `book` object.
@app.post("/books")
def create_book(book: Book):
    # book is a validated Book instance; access fields with dot notation.
    return {"message": f"Created '{book.title}' by {book.author}", "data": book}
```

Send this JSON body to `POST /books`:

JSON

```
{"title": "Dune", "author": "Frank Herbert", "year": 1965, "genre": "scifi"}
```

...and you get a clean confirmation. Send `"year": "long ago"` and FastAPI rejects it with a `422` pinpointing the bad field — again, no validation code from you.

Combining path + query + body in one endpoint

FastAPI figures out where each argument comes from by where it appears: in the path → path param; a Pydantic model → body; anything else → query param. You can mix all three:

PYTHON

```
@app.put("/users/{user_id}/books/{book_id}")
def update_user_book(
    user_id: int,          # path parameter
    book_id: int,          # path parameter
    book: Book,            # request body (Pydantic model)
    notify: bool = False,  # query parameter (?notify=true)
):
    return {"user_id": user_id, "book_id": book_id, "notify": notify, "book": book}
```

TIP

Tip — the mental rule: *In the path string → path param. A Pydantic model (or `dict`) → request body. Everything else → query param.* Memorise this and you'll always know how to read a parameter's source at a glance.

COMMON MISTAKES

Common Mistakes: Trying to send a request body with a `GET` request. While technically possible, it's against convention and many tools (and FastAPI's docs UI) won't help you do it. If you need to send data, use `POST` / `PUT` / `PATCH`.

2.4 Headers and Cookies

Sometimes information rides along in the request **headers** or **cookies** rather than the path, query, or body. The classic example is an API key sent in a custom header.

Reading headers with `Header()`

PYTHON

```
from fastapi import FastAPI, Header

app = FastAPI()

# FastAPI converts the argument name user_agent → the header "User-Agent"
# automatically (underscores become hyphens). x_api_key → "X-API-Key".
@app.get("/headers-demo")
def headers_demo(
    user_agent: str | None = Header(default=None),
    x_api_key: str | None = Header(default=None),
):
    return {"user_agent": user_agent, "x_api_key": x_api_key}
```

If a client sends the header `X-API-Key: secret123`, the `x_api_key` argument receives `"secret123"`. (Note the automatic underscore-to-hyphen conversion — a small but handy convenience.)

Reading cookies with `Cookie()`

PYTHON

```
from fastapi import FastAPI, Cookie

app = FastAPI()

@app.get("/cookies-demo")
def cookies_demo(session_id: str | None = Cookie(default=None)):
    return {"session_id": session_id}
```

TIP

Tip — a common real use case: A simple API-key check reads `x_api_key` from the header and compares it to an expected value, returning `401 Unauthorized` if it's missing or wrong. We'll build proper, production-grade authentication in Phase 5 — for now, just know that `Header()` is how raw header values reach your function.

2.5 Response Customisation

By default FastAPI takes whatever you return and sends it as JSON with a `200` status. But you can take full control of the response when you need to.

Different response types

PYTHON

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse, HTMLResponse, PlainTextResponse

app = FastAPI()

@app.get("/html", response_class=HTMLResponse)
def get_html():
    return "<h1>Hello in HTML</h1>"      # sent with Content-Type: text/html

@app.get("/text", response_class=PlainTextResponse)
def get_text():
    return "Just plain text"             # sent with Content-Type: text/plain

@app.get("/custom-json")
def custom_json():
    # Return a JSONResponse directly to control status code and headers.
    return JSONResponse(
        content={"message": "made manually"},
        status_code=201,
        headers={"X-Custom-Header": "hello"},
    )
```

Returning files and streaming

For sending a file from disk, use `FileResponse`; for large or generated-on-the-fly data, use `StreamingResponse`:

PYTHON

```
from fastapi import FastAPI
from fastapi.responses import FileResponse, StreamingResponse

app = FastAPI()

@app.get("/download")
def download():
    # Serves a file from disk with the right headers for download.
    return FileResponse("report.pdf", media_type="application/pdf",
                        filename="report.pdf")

@app.get("/stream")
def stream():
    # A generator yields chunks; useful for big files or live data.
    def row_generator():
        for i in range(5):
            yield f"row {i}\n"
    return StreamingResponse(row_generator(), media_type="text/plain")
```

TIP

Tip: Reach for these only when you need them. For the vast majority of endpoints, returning a dict or a Pydantic model and letting FastAPI serialize it to JSON is exactly right — simpler is better.

CONCEPT CHECK

Concept Check: You want an endpoint that returns a downloadable CSV the client can save. Which response class fits? Answer: *FileResponse* if the CSV already exists on disk, or *StreamingResponse* if you're generating the rows on the fly.

HANDS-ON

Hands-On Exercise 2: Build the Book Library API (in-memory)

Now combine everything into the first real version of the **Book Library API** — the project you'll grow for the rest of the tutorial. For now, store books in a plain Python list in memory (we'll swap in a real database in Phase 4).

Requirements:

TEXT

POST	/books	→ create a book (title, author, year, genre)
GET	/books	→ list all books (optional filter by genre, sort by year)
GET	/books/{book_id}	→ get one book (404 if not found)
PUT	/books/{book_id}	→ update a book (404 if not found)
DELETE	/books/{book_id}	→ delete a book (404 if not found)

Here is a complete, working solution:

```

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel

app = FastAPI(title="Book Library API")

# ---- The data shape clients send when creating/updating a book ----
class BookCreate(BaseModel):
    title: str
    author: str
    year: int
    genre: str

# ---- The data shape we send back (includes the server-assigned id) ----
class Book(BookCreate):
    id: int

# ---- In-memory "database": a list and a counter for ids ----
books: list[Book] = []
next_id = 1

@app.post("/books", response_model=Book, status_code=status.HTTP_201_CREATED)
def create_book(payload: BookCreate):
    global next_id
    # Build a full Book by combining the client's data with a new id.
    book = Book(id=next_id, **payload.model_dump())
    books.append(book)
    next_id += 1
    return book

@app.get("/books", response_model=list[Book])
def list_books(genre: str | None = None, sort_by_year: bool = False):
    result = books
    # Optional filter by genre (a query parameter).
    if genre is not None:
        result = [b for b in result if b.genre == genre]
    # Optional sort by year (a boolean query parameter).
    if sort_by_year:
        result = sorted(result, key=lambda b: b.year)
    return result

def _find_index(book_id: int) -> int:
    # Helper: return the list index of a book, or raise 404 if missing.
    for i, b in enumerate(books):
        if b.id == book_id:
            return i
    raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                        detail=f"Book {book_id} not found")

@app.get("/books/{book_id}", response_model=Book)

```

```
def get_book(book_id: int):
    return books[_find_index(book_id)]

@app.put("/books/{book_id}", response_model=Book)
def update_book(book_id: int, payload: BookCreate):
    i = _find_index(book_id)
    # Replace the whole record but keep the same id (PUT = full replace).
    updated = Book(id=book_id, **payload.model_dump())
    books[i] = updated
    return updated

@app.delete("/books/{book_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_book(book_id: int):
    i = _find_index(book_id)
    books.pop(i)
    # 204 = success, no body to return.
    return None
```

Test the flow in `/docs`:

TEXT

```
POST /books {"title":"Dune","author":"Herbert","year":1965,"genre":"scifi"}
    → 201, returns the book with "id": 1
GET /books?genre=scifi&sort_by_year=true → 200, list containing Dune
GET /books/1 → 200, the Dune book
GET /books/999 → 404, {"detail":"Book 999 not found"}
PUT /books/1 {"title":"Dune","author":"Frank Herbert","year":1965,"genre":"scifi"}
    → 200, updated book
DELETE /books/1 → 204, no body
GET /books/1 → 404 (it's gone)
```

What just happened? You used all three input channels in one small app: the **path** (`/books/{book_id}`) to identify a specific book, **query parameters** (`?genre=&sort_by_year=`) to filter and sort the list, and the **request body** (a `BookCreate` model) to carry new data. You returned the correct status codes — `201` on create, `204` on delete, `404` when something's missing — and used a Pydantic `response_model` so the docs know exactly what each endpoint returns. This is a genuinely RESTful CRUD API. The only thing keeping it from being "real" is that the data lives in memory and vanishes on restart — which is exactly what Phase 4 fixes.

COMMON MISTAKES

Common Mistakes: Forgetting that the in-memory `books` list is wiped every time the server reloads. If your data "disappears," you probably saved a file (triggering `--reload`) or restarted uvicorn. That's expected here — persistence comes in Phase 4.

TIP

Tip: Notice the model pattern: `BookCreate` (what the client sends) and `Book` (what we store and return, with the server-assigned `id`). Separating "input shape" from "stored/output shape" is a habit that pays off enormously — Phase 3 formalises it into a clean inheritance hierarchy.

Checkpoint: At This Point You Can Now...

- ☐ Declare and validate **path parameters**, including typed and `Enum`-restricted ones.
- ☐ Explain and avoid the **route-ordering bug** (specific routes before parameterised ones).
- ☐ Use **query parameters** for filtering/sorting/pagination, with required, optional, default, boolean, and list variants.
- ☐ Accept a **request body** with a Pydantic model and explain why that beats a raw dict.
- ☐ **Combine** path, query, and body in a single endpoint and know where each value comes from.
- ☐ Read **headers** with `Header()` and **cookies** with `Cookie()`.
- ☐ Customise responses with `HTMLResponse`, `PlainTextResponse`, `JSONResponse`, `FileResponse`, and `StreamingResponse`.
- ☐ Build a full **CRUD** API with correct status codes and `404` handling.

Phase 2 Summary

PHASE 2 • KEY TAKEAWAYS

In this phase your endpoints came alive — they now accept and respond to real input.

- **Path parameters** (`/books/{book_id}`) identify a specific resource; type hints validate them; declare **specific routes before parameterised ones**.
- **Query parameters** are any non-path arguments (`?genre=scifi`); use them to filter, sort, and paginate. Defaults make them optional; `Query()` adds rules and list support.
- The **request body** carries structured JSON for create/update; a **Pydantic model** parses and validates it for you — far better than a raw `dict`.
- FastAPI decides each argument's source automatically: **in the path → path param, Pydantic model → body, everything else → query param**.
- `Header()` and `Cookie()` read those parts of the request; response classes (`HTMLResponse`, `FileResponse`, `StreamingResponse`, ...) customise what you send back.
- **You built a full CRUD Book Library API** with all three input channels, proper status codes, and `404` handling — your first genuinely RESTful service.

You can now build dynamic, parameterised endpoints. Next phase, you'll go deep on Pydantic to make your data validation truly iron-clad.

End of Phase 2. Reply "**continue**" for Phase 3 — Pydantic: Data Validation — where you'll turn today's quick taste of models into bulletproof validation with custom validators, nested models, and clean response schemas.